

Resumen Ejecutivo

Supabase es una plataforma de backend open-source basada en PostgreSQL que ofrece autenticación, API instantánea, funciones Edge, suscripciones en tiempo real y almacenamiento de archivos ¹ ². Para construir aplicaciones empresariales o ERPs escalables con Supabase se aprovechan sus componentes integrados (Autenticación, Base de datos, Realtime, Almacenamiento, Functions) y bibliotecas/servicios externos compatibles. En este informe analizamos bibliotecas de UI, formularios, tablas de datos, graficado, sistemas de autenticación/SSO, gestión de archivos, módulos de reportes/facturación, además de proyectos y buenas prácticas de arquitectura (diseño de esquema, seguridad por niveles, migraciones, multi-tenancy, caching, backups, observabilidad, despliegue). Incluimos ejemplos de código, diagramas de arquitectura (Mermaid) y tablas comparativas para ayudar al desarrollo de ERPs escalables con Supabase.

Bibliotecas de UI y componentes

- **Supabase UI (shadcn)**: Librería oficial basada en [shadcn/ui](#) y Tailwind CSS. Provee componentes listos para autenticación, upload, chat, avatar, tablas, etc. Está integrada con Supabase Auth, Storage y Realtime ¹. Mantenibilidad media (reciente en 2025), licencia MIT, código en el repositorio [supabase/ui](#) (archivado) y documentación en el blog de Supabase. Ejemplo: [Next.js Starter de Supabase](#) utiliza [supabase/ui](#) para flujos de login y elementos de UI ¹.
- **supabase-community/auth-ui**: Conjunto de componentes React preconstruidos para Autenticación Supabase. Código abierto MIT ([supabase-community/auth-ui](#), archivado octubre 2025). Facilita flujos de registro y sesión con Supabase Auth. Está documentado en la guía oficial “Auth Helpers” de Supabase. Ejemplo: usado en proyectos que requieren autenticación sin diseñar formularios desde cero.
- **Supabase Grid (@supabase/grid)**: Componente React para mostrar y editar tablas de Postgres. Era usado en el dashboard de Supabase ³. Licencia MIT. *Nota*: este repositorio fue deprecado y migrado al repo principal de Supabase. Integra Supabase Realtime para refrescar datos. Un uso típico es montar `<SupabaseGrid table="..." />` pasando funciones para ejecutar consultas SQL ⁴.
- **@supabase/react-data-grid**: Antiguo grid (basado en [React-Data-Grid](#)). Versión beta (última v7.1 en 2022) MIT, pero poco activo. Se requerirían instalar peer deps (React, react-dom, supabase/ui). Hoy en día se prefiere ag-Grid, MUI DataGrid u otros.
- **UI Genéricas**: Muchas librerías populares funcionan con Supabase. Ejemplos: **Material-UI (MUI)**, **Chakra UI**, **Mantine**, **Ant Design**, **Radix UI**. No tienen integración específica con Supabase, pero se usan para formularios y tablas. La licencia varía (MIT o similar). Usar estas librerías para componentes de UI es común.
- **shadcn/ui**: Colección de componentes UI personalizables basada en Radix y Tailwind. Aunque no es “de Supabase”, es usada por [supabase/ui](#) ¹. Licencia MIT. Ideal para dashboards y formularios complejos.
- **React Hook Form / Formik / Zod**: Librerías de manejo de formularios/validación en frontend. No son específicas de Supabase, pero se combinan con [supabase-js](#) en apps React/Next. Ejemplo: el [School ERP Template](#) usa React Hook Form + Zod para validar formularios con Supabase.

- **Constructores de formularios:** *SupaForm* (supaform.io) es una herramienta web (no OSS) que crea formularios drag&drop conectados a Supabase. Conecta a tu proyecto Supabase y genera tablas/RLS automáticamente. Plan freemium, sin repositorio abierto. Útil para prototipos rápidos de entrada de datos sin codificar formularios.

Librerías de tablas de datos (Data Grids)

- **supabase/grid** (anterior): Ya mencionado en UI.
- **AG Grid:** Solución robusta (versión Community gratuita MIT, Enterprise de pago). Soporta grandes volúmenes, filtrado, agrupamientos. Integración: fetch de datos con supabase-js (Postgres queries) para poblar las filas. Alto rendimiento, bien mantenido. Documentación en ag-grid.com. Ejemplos: “Data Grid con Supabase y React” (p. ej. usando `supabase.from('tabla').select()` para llenar datos).
- **MUI DataGrid:** Versión básica MIT, Pro de pago. Creada por Material-UI. Integración similar: consultas a Supabase. Ventajas: integrado a ecosistema MUI, fácil tematizar. Contras: en versión libre limitado en features (p. ej. row grouping en Pro).
- **TanStack Table (React Table):** Librería headless (MIT) para construir tablas. Muy flexible, pero requiere UI propio. Conecta con Supabase obteniendo datos, usa hooks para manejo.
- **Grid.js:** Tabla ligera (MIT) de JavaScript que se integra en cualquier frontend (React, Svelte). Puede usar llamadas a Supabase REST. Menos popular pero interesante.
- **Tabulator:** Data grid (MIT) con muchas funciones (edición, filtrado, exportación). Se integra via JS o React wrappers. Útil para dashboards ERP que necesitan edición en sitio.
- **DevExtreme DataGrid:** Comercial (DevExpress). Altamente personalizable. Licencia propietaria. Integra llamadas a Postgres o API.
- **SupaGrid:** No existe un proyecto oficial con este nombre (solo supabase/grid).

Bibliotecas de gráficos (Charting)

Las apps de ERP suelen requerir gráficos de desempeño, ventas, etc. Supabase puede alimentar estas con consultas SQL. Librerías populares:

- **Chart.js** (MIT): Fácil, 2D charts (barras, líneas, pasteles). Con Supabase, obtener datos y pasar a `new Chart(ctx, { ... })`. Ejemplo: “chart con Supabase y Chart.js”.
- **Recharts** (MIT): Componentes React para charts (basado en D3). Integración: `<BarChart data={data} ...>` donde `data` proviene de `supabase.from().select()`.
- **ECharts** (Apache 2.0): Librería de Baidu, muy completa. Permite visualizaciones interactivas. Su curva de aprendizaje es más pronunciada. Se conecta igual con datos de Supabase.
- **ApexCharts** (MIT + Pro): Librería JS con wrappers para React/Vue. Fáciles de integrar.
- **D3.js** (BSD-3): Poderosa, pero de bajo nivel (requiere montar gráficos manualmente). Con Supabase se puede usar D3 para hacer gráficos custom. Licencia BSD.
- **Google Charts / Highcharts:** Variantes para web. Licencia propietaria (gratuito con condiciones). Se pueden usar conectando con Supabase.
- **Plotly.js** (MIT): Otro library JS; para ERPs puede generar dashboards financieros interactivos. Requiere dependencias grandes.

(No se han encontrado componentes de gráficos específicos de Supabase, ya que la integración es genérica: usar supabase-js para consultas y alimentar la librería de gráficos).

Autenticación y SSO

- **Supabase Auth:** Servicio nativo. Maneja usuarios, sesiones, OAuth con Google, GitHub, Apple, Azure AD, etc. Permite SSO empresarial (Planes Pro) vía SAML con proveedores (Auth0, Okta, OneLogin, Azure) ⁵. Los datos de sesión se exponen a RLS para seguridad.
- **supabase/auth-helpers:** SDKs oficiales (JS, Python, Flutter, etc.) para integrar Auth en distintas plataformas.
- **Clerk:** Servicio de autenticación (OAuth, Múltiples factores, Billing). Puede funcionar con Supabase usando JWTs. Supabase docs explican integración de Clerk.
- **Auth0:** Integración oficial en docs para usar Auth0 como auth externa de Supabase. Usar JWTs de Auth0 en RLS.
- **AWS Cognito:** Soportado (incluso en docs internos).
- **WorkOS:** Plataforma SSO (Google Workspace, Okta, etc.) integrable a Supabase (ver docs "WorkOS Integration").
- **NextAuth.js:** SDK de auth para Next.js. Puede combinar Supabase (DB) con proveedores (GitHub, Google). No oficial de Supabase, pero común en proyectos Next.js con Supabase.
- **Ory/Kratos:** Sistema de identidad de código abierto. Puede usarse con Supabase almacenando identidades en Postgres.
- **Keycloak:** Servidor de SSO de código abierto. Podría usarse junto a Supabase (ej. usuarios en Keycloak, luego sincronización a supabase).
- **Pangea:** Herramienta de OTP/SMS/email. Integrable generando tokens en Functions.
- **MDM Corporativos:** Okta, OneLogin, Azure AD se conectan mediante SAML nativo de Supabase ⁵. Por ejemplo, Okta+Supabase SAML es una opción enterprise.
- **Tarjeta/2FA externas:** como Twilio Verify, Authy (solo relevancia si se necesita 2FA avanzada fuera de lo que ofrece Supabase).

Tabla: Comparativa de soluciones de Autenticación

Herramienta	Integración Supabase	Pros	Contras	Licencia/Servicio
Supabase Auth	Nativa (Auth API, SDK)	Fácil de usar, RLS integrado, OAuth social	Planes limites en free	Gratis/Pro
Clerk	Supported (docs oficial)	UI preconstruida, SSO, licencias subcripciones	Costo, vendor-lockin	SaaS (planes)
Auth0	Integración doc oficial	SSO maduro, 2FA, extensible	Costo, aumenta complejidad	SaaS (planes)
AWS Cognito	Soporte (Amplify PWDS)	Se integra a AWS, escalable	Configuración compleja, lock AWS	Gratuito con AWS servicios
Keycloak	Indirecto (HTTP/pg)			

Herramienta	Integración Supabase	Pros	Contras	Licencia/Servicio
Open-source, SSO corporativo	No nativo Supabase, mantener tú mismo	Apache 2.0		
NextAuth.js	Con Supabase adapter (comunitario)	Popular con Next, multiópt	No oficial, depende de plugins	MIT

(Tabla de ejemplo; ver documentación oficial para detalles específicos)

Trabajos en segundo plano y colas

Supabase no tenía colas nativas, pero recientes actualizaciones incluyen **Supabase Cron** y **Queues (pgmq)** como parte del stack Postgres:

- **Supabase Cron:** Módulo de Postgres (extensión pg_cron) para tareas programadas ⁶. Permite crear jobs periódicos usando sintaxis cron (desde cada segundo hasta anual) ⁶. Los jobs pueden ejecutar SQL o llamar funciones Edge o webhooks ⁷. Hay consola gráfica para monitorear ejecuciones. Según la documentación: “Supabase Cron simplifica el scheduling de jobs recurrentes... los Jobs pueden ejecutar SQL o funciones Edge” ⁶. Ideal para tareas de mantenimiento, envíos de email programados, etc.

- **Supabase Queues (pgmq):** Extensión Postgres que implementa colas de mensajes ligeras ⁸. Características clave: **“pgmq es una cola de mensajes ligera basada en Postgres, sin dependencias externas, entrega *exactly-once*, con API similar a AWS SQS”** ⁸. Se habilita creando la extensión (`CREATE EXTENSION pgmq;`). Permite `pgmq.send`, `pgmq.receive`, `pgmq.consume`, etc., todo dentro de la DB. En conjunción con Edge Functions y Cron, permite armar pipelines asíncronos (por ejemplo, en un Edge Function leer mensajes de la cola con `SELECT * FROM pgmq.receive(...)`).

- **Supabase Edge Functions:** Pueden usarse como workers. Un patrón común es: al insertar en cola (`pgmq.send` en un trigger o API), un Cron job dispara una Function que consuma la cola. Edge Functions también pueden invocarse externamente (HTTPS) para procesamiento intensivo.

- **Soluciones externas:** Para necesidades muy complejas, se pueden integrar colas externas (RabbitMQ, BullMQ con Redis, Celery con Rabbit/Redis, Temporal.io). Por ejemplo, usar **BullMQ** (Node.js) apuntando a un servidor Redis para colas de tareas, donde los triggers de Supabase (INSERT) envían mensajes a Redis. O bien **Trigger.dev** (servicio de workers en la nube) disparados por cambios en Supabase. Estas integraciones son genéricas, no específicas de Supabase.

- **Cron jobs tradicionales:** Con Supabase CLI o una instancia externa (podríamos usar DigitalOcean Cron o cron de Linux) que invoque APIs de Supabase a intervalos. Menos integrado que Supabase Cron, pero válido.

- **Tareas por filas (row-level):** Usar triggers/PostgreSQL triggereado para hacer trabajos rápidos dentro de la DB. Luego limpiar con vacuum o jobs.

Almacenamiento de archivos y adjuntos

- **Supabase Storage:** Solución nativa (compatible S3) para manejar archivos (imágenes, PDFs, documentos) ⁹. Ofrece buckets con CDN global y procesamiento de imágenes integrado ¹⁰ ⁹.

Control de acceso mediante RLS por objetos: se define RLS en las tablas de metadatos de Storage, permitiendo permisos granulares ¹⁰. Se accede en código con

```
supabase.storage.from('bucket').upload(path, file, {metadata}). Funciona en frontend y backend.
```

- **Cientes de upload:** Librerías JS como [Dropzone](#) o [Uppy](#) se integran con Supabase Storage para subir múltiples archivos con UI arrastrar& soltar. En Supabase UI Library hay un componente *File Upload Dropzone* listo.
- **Otras opciones de almacenamiento:** Podríamos usar directamente AWS S3, Google Cloud Storage, Cloudinary o Filestack. Integración: almacenar credenciales y usar SDKs (no recomendado duplicar con supabase storage, salvo casos de imágenes optimizadas específicas). Si se usa Supabase, **no se necesita otro almacenamiento** en la mayoría de casos.
- **Previsualización y Procesamiento:** Cloudinary o imgix ofrecen procesamiento avanzado de imágenes, pero Supabase Storage ya tiene image transforms. Puede usarse si hay requisitos muy específicos (por ejemplo, reconocimiento de objetos).
- **Bases de datos de archivos:** Algunas apps usan tablas Postgres con campos BYTEA para adjuntos, pero con supabase storage es más común guardar URL en DB y archivo en bucket.

Módulos de reportes y facturación

No existen “módulos ERP integrados” específicos de Supabase por defecto. Sin embargo, hay herramientas que se conectan a Postgres para reportes:

- **Metabase:** Herramienta OSS de BI. Supabase incluso tiene docs dedicadas ¹¹: “Metabase es una herramienta open source de visualización. Puedes usarla para explorar datos en Supabase” ¹¹. Se conecta por la cadena de conexión Postgres. Permite dashboards personalizados. Ideal para reportes financieros o métricas de ERP.
- **Apache Superset:** Dashboards avanzados con soporte de consultas SQL. Licencia Apache. Se configura sobre la base de datos de Supabase.
- **Grafana:** Enfocado en métricas/tiempos. Tiene plugin Postgres. Se puede usar para monitoreo de la base de datos o rep.
- **Redash:** Precursor de Metabase, ofrece consultas SQL y visualización.
- **Microsoft PowerBI/Tableau:** Conectar vía ODBC/JDBC (Postgres) a Supabase para reportes empresariales (son cerrados/pro).
- **Reportes personalizados:** Usar librerías para generar PDFs/Excel. Ejemplos: - **jsPDF** o **pdfmake** (JavaScript): generar PDFs en frontend/Edge. - **PDFKit** (Node.js): generar en backend/Edge. - **puppeteer** (Node): renderizar HTML/CSS y exportar PDF (útil para facturas con diseño). - **Docxtemplater** o **xlsx-populate**: generar documentos Word/Excel.

Patrón común: una función serverless lee datos (ej. una orden desde Postgres), formatea HTML (o datos JSON) y genera el PDF con alguna librería; luego lo almacena en Supabase Storage o lo envía por email. Por ejemplo, “*Invoice generation with Node.js*”. Aunque no hay una librería Supabase específica para facturas, estas soluciones externas son estándar.

- **ERP específicos:** Algunos proyectos de ERP incluyen módulos de facturación interna, pero son personalizados. Un ejemplo podría ser crear una tabla `invoices` en Supabase y construir vistas/ funciones para crear PDF. No hay un paquete preempaquetado, se suele desarrollar a medida.

Proyectos, plantillas y ERPs basados en Supabase

- **Carbon:** [crbnos/carbon](https://crbnos.com/carbon) es un ERP/MES/QMS open-source para manufactura. Usa Supabase como backend (DB y Auth) ¹². Tecnología: React+TypeScript (Radix UI, Tailwind), Edge Functions, RLS, caching (Upstash Redis). Funcionalidades: finanzas, MRP, BoM anidadas, trazabilidad, permisología avanzada, ABAC, roles (cliente/empleado/proveedor) ¹³ ¹⁴. Licencia MIT ¹⁵. Con 1.8k★, está en producción en empresas. Totalmente orientado a manufactura compleja.
- **HRM SaaS Boilerplate (phamvuhoang):** [nextjs-supabase-boilerplate](https://nextjs-supabase-boilerplate.com/) es un sistema de gestión de recursos humanos tipo SaaS. Características: gestión de empleados, contratos, nómina, calendario laboral, registro de horas, proyectos, clientes, asignación de habilidades ¹⁶ ¹⁷. Tech: Next.js 15 (App Router), Supabase (Auth, Postgres), Tailwind, shadcn/ui, React Context. Soporta multi-tenancy (cada “empresa” con sus datos aislados). Perfecto como punto de partida para ERP pequeño enfocándose en RRHH.
- **School Management ERP:** [akashmahlaz/School-Management-System](https://akashmahlaz.com/School-Management-System) – Plantilla de gestión escolar en Next.js + Supabase. Funciones: admisiones, gestión de clases, notas, dashboard (gráficos Recharts), UI con shadcn/ui ¹⁸. Muy básico pero un ejemplo de ERP para educación.
- **SaaS Starter Kits:** Varios boilerplates para apps SaaS/E-commerce que incluyen supabase y facturación:
 - *SaaS-Starter-Kit* (React, Next): [Saas-Starter-Kit/Saas-Kit-supabase](https://saas-starter-kit.com/) – cuenta con autenticación, dashboard admin, Stripe (pagos por suscripción), etc ¹⁹ ²⁰. Licencia MIT. Gran cantidad de ejemplos de flujos SaaS.
 - *Stripe-Supabase SaaS Template* (dzlau): [dzlau/stripe-supabase-saas-template](https://dzlau.com/stripe-supabase-saas-template) – Next.js + Drizzle ORM + Stripe, incluye OAuth, Stripe Checkout y Customer Portal para suscripciones ²¹.
 - *CMSaaS Starter* (SvelteKit): [scosman/CMSaaSStarter](https://scosman.com/CMSaaSStarter) – SaaS completo con marketing, blog, pagos Stripe, diseño responsivo ²². Usa Supabase, Tailwind.
 - *Next.js + Supabase + Clerk* (JSM tutorial): [adrianhajdin/saas-template](https://adrianhajdin.com/saas-template) – plantilla educativa con Clerk (auth & billing) y Supabase ²³.
- *Next.js + Supabase + Stripe* (KolbySisk): [KolbySisk/next-supabase-stripe-starter](https://kolbysisk.com/next-supabase-stripe-starter) – starter kit de alta calidad con Stripe, Suscripciones, Resend (emails), y Supabase migraciones ²⁴. Estos kits no son ERPs empresariales completos, pero contienen módulos de facturación, auth y dashboards típicos de apps empresariales.
- **Plantillas de facturación:** No hay una “aplicación de facturación” oficial con Supabase, pero es común integrar Stripe u otros. Por ejemplo, el sitio de Mercado Pago tiene guías para Webhooks (ver [85†L5-L12]), pero es más externo. En la comunidad, algunos repositorios abordan el billing con Stripe en Supabase (ver enlaces de **Stripe Starter Kits** arriba).

Buenas prácticas de arquitectura escalable con Supabase

- **Diseño del esquema:** Para multi-tenant se aconseja un único esquema (p. ej. “public”) con una columna `tenant_id` en cada tabla relevante. Cada índice debe incluir `tenant_id` como primer campo para que las consultas estén naturalmente filtradas ²⁵. Alternativa (menos común): múltiples esquemas por cliente, pero complica mantenimiento ²⁶. El enfoque estándar es una DB, RLS por `tenant_id`. Los expertos han escalado con 2M de clientes y 10B de filas usando este patrón ²⁵. Siempre habilitar Row-Level Security (RLS) y políticas restrictivas ² ²⁷. Se suele definir funciones PL/pgSQL `policy` para filtrar por `auth.uid()` y `tenant_id`.

- **RLS y seguridad:** Habilitar RLS en todas las tablas accesibles públicamente ². Las políticas (`CREATE POLICY`) actúan como filtros automáticos en cada SELECT/INSERT/UPDATE. Se recomienda combinar RLS con roles personalizados en JWT para aislar empresas. Supabase Docs advierten: “RLS es increíblemente poderoso... proporciona seguridad de extremo a extremo junto con Supabase Auth” ²⁷. En ambiente de producción, verificar que TODO dato sensible tenga RLS activa ²⁸. Por ejemplo, la lista de control de producción sugiere “**RLS habilitado en todas las tablas**” antes de lanzar ²⁸.
- **Migraciones:** Usar **Supabase CLI** (o adaptadores de herramientas ORM como Drizzle, Prisma) para migraciones gestionadas por SQL. Mantener las migraciones en control de versiones y aplicarlas en orden en staging/producción ²⁹. Se recomienda probar migraciones en staging antes de prod ²⁹. El repositorio del proyecto debe incluir scripts de migración y seeds. Supabase ofrece la interfaz de migraciones en CLI y soporte de monitor para errores. Documentar y versionar cambios de esquema.
- **Multi-tenancy:** Como vimos, RLS + tenant_id es la vía más escalable. Otra opción es **particionar tablas por tenant** (particiones Postgres) manteniendo un esquema común. Cada partición actúa como tabla separada en disco ³⁰. Esto mejora rendimiento físico sin reesquematizar. Pero, en general, implementar RLS con `tenant_id` en cada consulta es suficiente. Avoid multi-schema unless regulaciones requieren separación completa ³¹.
- **Uso de Realtime:** Supabase Realtime (canales WebSocket) permite suscribirse a cambios en tablas. Para ERPs, útil en dashboards en vivo (p. ej. actualizar inventario o notificaciones). Pero tener en cuenta: cada suscripción mantiene conexión; demasiadas puede saturar. Lo ideal es limitar suscripciones a vistas cruciales. Usar filtros en `.on()` para no recibir exceso de datos. RLS también se aplica a Realtime.
- **Caché:** Supabase no provee caché integrado. Para cargas elevadas, implementar cache en capa front/backend. Ejemplos: usar Redis o Upstash (como en Carbon) para resultados de consultas intensivas (inventarios, cálculos históricos). Un patrón: Edge Function consulta datos y guarda en Redis con TTL, el front verifica cache antes de llamar DB. Cloudflare/CloudFront o Vercel edge cache también pueden almacenar respuestas.
- **Backups y recuperación:** Supabase Pro incluye backups diarios automáticos ³². Sin embargo, se recomienda exportar datos críticos periódicamente (por ejemplo, con una función Edge que haga `pg_dump` programado) ³³. Probar restauraciones regularmente. En plan gratuito o self-hosting, configurar cron externo para dumps y almacenarlos fuera. Tener plan de recuperación ante desastres documentado ³⁴.
- **Observabilidad y monitoreo:** Registrar acciones en tablas de auditoría (logs propios). Usar **pg_stat_statements** (habilitable en Supabase) para métricas de consultas lentas. Capturar logs de errores en el nivel de DB y Application (Edge Functions usan consola). Supabase Dashboard ofrece logs de Realtime/Edge. Para monitoreo avanzado, conectar DB a sistemas como Datadog o Prometheus (usando extensiones Postgres o logs). Supabase Status (sistema) para infra.
- **Despliegue y ambientes:** Separar entornos: producción y staging en proyectos distintos. Conectar con [Supabase CLI link](#) para apuntar localmente. Nunca exponer keys de servicio en frontend; mantén las variables en env vars y secretos. Seguir checklist de producción ²⁸ (p.ej. revisión de RLS, claves seguras, SSL activo, Términos de servicio). Versionar infraestructura si se usa Terraform.
- **Escalabilidad de DB:** Elegir un plan acorde (supabase ofrece escalado vertical con CPUs/ram y réplicas de lectura). Para mucha carga, usar read replicas para reportes/analytics, dejando transacciones en la primaria ³⁵. Considerar particionamiento de tablas (Postgres partitioning) si crecen excesivamente.

Tabla: Prácticas de base de datos en ERP con Supabase

Aspecto	Recomendación	Cita/Detalles
Esquema (Multi-tenant)	Columna <code>tenant_id</code> + un solo esquema	Índices con <code>tenant_id</code> en primer lugar ²⁵ ; RLS por tenant.
Seguridad (RLS)	Habilitar RLS en todas las tablas expuestas	"RLS debe estar habilitado siempre..." ² . Uso de <code>auth.uid()</code> .
Migraciones	Versionar migraciones, probar en staging antes de prod	"Aplicar migraciones a staging, probar, luego a producción" ²⁹ .
Backups	RespalDOS diarios (plan Pro) + exportaciones externas	"Supabase Pro incluye backups diarios; exportar externamente" ³⁶ .
Caché	Capas de caché (Redis, CDN) para queries intensivas	<i>No hay cita directa, pero es buena práctica estándar.</i>
Observabilidad	Tablas de auditoría, <code>pg_stat_statements</code> , logs	"Loggear acciones en tabla de auditoría" (Leanware consejos).
Despliegue	Entornos separados (prod, staging), CI/CD de migraciones	"Proyectos separados, <code>supabase link</code> , CI ejecuta migraciones" ²⁹ .

Pilas recomendadas y arquitecturas de ejemplo

Para ERPs con Supabase se suele usar stacks modernos de frontend y despliegue en la nube. A continuación sugerimos algunas configuraciones:

- **Frontends comunes:**
- **React** (Next.js 14/15 con App Router): Permite SSR/SSG y usar `@supabase/auth-helpers-nextjs` para SSR seguro.
- **Vue.js / Nuxt 3:** Con Supabase JS SDK o `@supabase/supabase-js`.
- **SvelteKit:** Trabajo reciente (p.ej. [CMSaasStarter](#)).
- **Angular:** Puede usar `supabase-js` en servicios.
- **Frameworks móviles:** Flutter, React Native, Ionic: Supabase tiene SDKs oficiales (Flutter y JS).
- **Gestión de estado:** En React: Redux Toolkit, React Query/TanStack Query, Recoil, o la context API simple. Para mantener datos de tablas/proyectos, React Query con `mutate`/suscribir es útil.
- **Deploy:**
- **Vercel / Netlify / Fly:** Para Next.js/React, ofrecen integración con Supabase (variables de env, redeploy automático).
- **Render.com:** Configurable, soporta Docker.
- **DigitalOcean App Platform:** También integrado con Supabase.
- **Cloudflare Pages/Workers:** Para SPAs estáticas + Cloudflare Workers para SSR o Functions, junto con Supabase.
- **Kubernetes / Docker:** Si se requiere autoscaling granular, desplegar frontends en contenedores y escalar con K8s.

- **Bases de datos y servicios:** Supabase es base de datos (DBaaS), no requiere instalar nada. Solo configurar en la nube.
- **CDN/caché:** Para assets estáticos y API, usar CDN (Vercel/Cloudflare). Supabase Storage ya usa CDN para archivos.
- **Monitorización:** Puede incluir NewRelic/Datadog en Edge Functions, y alertas de Supabase (logs + cron logs).
- **Mermaid – Ejemplo de arquitectura a pequeña escala** (por ejemplo, Pyme):

```

flowchart LR
    subgraph Cliente
        A[Frontend SPA (Next.js/Vue)]
    end
    subgraph BackendSupabase
        B[(Postgres)]
        C[Auth]
        D[Storage]
        E[Edge Functions]
        F[Realtime]
    end
    A -->|Auth| C
    A -->|API (postgrest)| B
    A -->|Storage SDK| D
    A -->|Sockets (Realtime)| F
    E -->|Funciones| B
    E -->|Funciones| D
    E -->|HTTP API| Clientes

```

- **Arquitectura mediana:**

```

flowchart LR
    subgraph Frontend
        A[Next.js 14 SSR/CSR]
        G[React Query/Redux]
    end
    subgraph SupabasePlatform
        B[(Postgres DB)]
        C[Supabase Auth]
        D[Supabase Storage]
        E[Edge Functions (FaaS)]
        F[Realtime (pouch)]
        Cron[Supabase Cron]
        Q[Queues (pgmq)]
    end
    subgraph Workers

```

```

    W[(Redis/Bull)]
end
subgraph External
    M[Stripe/PayPal]
    N[Stripe webhooks]
end
A -->|JWT| C
A -->|REST/API| B
A -->|Storage| D
A -->|WebSockets| F
A -->|Edge HTTP| E
E -->|DB Queries| B
E -->|Cache| W
Cron -->|SQL/Edge| B & E
B -->|Trigger| Q
Q -->|Pop| E
N --> E
M --> E

```

- **Arquitectura grande:** Considerar microservicios y escalamiento. Ejemplo con funciones y colas:

```

flowchart LR
    subgraph Clientes
        A1[WebApp (Next.js)]
        A2[Mobile App (React Native)]
    end
    subgraph InfraSupabase
        DB[(PostgresMaster)]
        DBrep[(PostgresReplica)]
        Auth[Auth Service]
        Stg[Storage Buckets]
        Realtime[Realtime Server]
        CF[Cloudflare Workers]
        Efn[Edge Functions]
        Cron[Cron Manager]
        QL[Queue/pgmq]
    end
    subgraph ServiciosExtras
        Redis[Redis Cache]
        Rabbit[Mensajería]
        DataEng[Data Warehouse]
    end
    A1 --> Auth
    A1 --> Realtime
    A1 --> DB
    A2 --> Auth

```

```
A2 --> DB
Auth --> DB
Efn --> DB
Efn --> Stg
Cron --> DB
QL --> Efn
Realtime --> DB
Redis -->|cache queries| DB
DB --> DataEng
```

Ejemplos de código e integración

• Flujos de autenticación:

```
// Frontend (React) con supabase-js
const { supabase } = useSupabaseClient();
const signUp = async (email, pass) => {
  let { user, error } = await supabase.auth.signUp({ email, password:
pass });
  // Enviar verificación al email...
};
const signIn = async (email, pass) => {
  let { user, error } = await supabase.auth.signInWithPassword({ email,
password: pass });
  // user contains uid y sesión
};
const { data: session } = supabase.auth.onAuthStateChange((event, session)
=> {
  // manejar cambios de sesión
});
```

• Subidas a Supabase Storage:

```
// Upload de archivo a bucket 'invoices'
const uploadFile = async (file) => {
  const { data, error } = await supabase.storage
    .from('invoices')
    .upload('files/' + file.name, file, {
      cacheControl: '3600',
      upsert: false
    });
  if (error) console.error(error);
  return data; // contiene path y metadata
};
// Para descargar:
```

```
const { data: fileData } = await supabase.storage
  .from('invoices')
  .download(data.path);
```

- **Suscripciones Realtime:**

```
// Escuchar inserciones en tabla 'orders'
supabase
  .channel('public:orders')
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table:
'orders' }, payload => {
    console.log('Nueva orden:', payload.new);
  })
  .subscribe();
```

- **Funciones Edge (Supabase):**

```
// /functions/invoice-generator
import { serve } from "https://deno.land/x/sift@0.5.0/mod.ts";
import pdfLib from 'pdf-lib'; // ejemplo de librería Deno
serve(async req => {
  const { invoiceData } = await req.json();
  // Generar PDF con pdf-lib o usar otra técnica
  const pdfBytes = await generateInvoicePDF(invoiceData);
  return new Response(pdfBytes, {
    headers: { 'content-type': 'application/pdf' }
  });
});
```

- **Ejemplo de RLS:**

```
-- Habilitar RLS en tabla "customers"
ALTER TABLE customers ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Customers solo accesibles a empresa"
  ON customers FOR SELECT
  USING (auth.jwt() ->> 'tenant_id' = customers.tenant_id::text);
```

- **Migraciones (Supabase CLI):**

```
supabase db diff --schema public > migrations/
2026_02_20_add_customers_table.sql
supabase db push
```

Resumen de Recursos Comunitarios

- **Repositorios Notables:**

- [crbnos/carbon](#): ERP/MES para manufactura con Supabase ¹² .
- [phamvuhoang/nextjs-supabase-boilerplate](#): HRM SaaS en Supabase ¹⁶ .
- [akashmahlaz/School-Management-System](#): Gestor escolar, Next.js+Supabase ¹⁸ .
- [Saas-Starter-Kit/Saas-Kit-supabase](#): Boilerplate SaaS (auth, Stripe) ¹⁹ .
- [dzlau/stripe-supabase-saas-template](#): Next.js SaaS starter con Stripe ²¹ .
- [scosman/CMSaaSStarter](#): SaaS template SvelteKit ²² .
- [KolbySisk/next-supabase-stripe-starter](#): Next.js+Supabase+Stripe profesional ²⁴ .
- **Plantillas oficiales:** ver [Vercel Examples](#) con Next.js + Supabase (autenticación, suscripciones, etc.).

- **Plugins y complementos:**

- [Authorizers para RLS](#) (libraries para reglas ABAC).
- [Supabase Studio Plugins](#) (aunque más orientado a extensión de la interfaz de Studio).
- [Supabase Terraform](#) (infra codificada).

- **Foros y comunidad:**

- Supabase Discord/Foros oficial.
- Temas en Reddit: *"Supabase multi-tenant best practices"* ²⁵ .
- Publicaciones en Dev.to (ej. *RLS en Supabase*).

- **Herramientas Visuales:**

- [SupaForm](#) (generador de formularios).
- [Strapi/Hasura](#) (no Supabase, pero alternativas de GraphQL que pueden conectarse a Postgres).
- [Metabase Templates](#) (plantillas de aplicaciones).

En conclusión, Supabase ofrece todos los bloques para construir un ERP moderno: base de datos SQL robusta, autenticación completa, almacenamiento y funciones sin servidor. La clave es diseñar bien el esquema con RLS, usar herramientas de UI y gestión de estado conocidas, y apoyarse en proyectos comunitarios como punto de partida. Las arquitecturas escalables combinan Supabase con deployment serverless o en contenedores, estrategias de caché y colas para carga de trabajo pesada, y monitoreo continuo. Con las integraciones mencionadas, es posible desplegar ERPs y sistemas de facturación de nivel empresarial sin depender de infraestructuras complicadas.

Fuentes: Documentación oficial de Supabase ⁶ ² ³⁷ , blogs de Supabase, repositorios GitHub públicos ¹² ¹⁹ y artículos técnicos relacionados ⁸ ¹¹ . (Las citas referencian los pasajes relevantes).

¹ GitHub - supabase/ui: Supabase UI Library

<https://github.com/supabase/ui>

² ²⁷ Row Level Security | Supabase Docs

<https://supabase.com/docs/guides/database/postgres/row-level-security>

³ ⁴ GitHub - supabase/grid: A react component to display your Postgresql table data. Used in Supabase Dashboard app.

<https://github.com/supabase/grid>

⁵ Single Sign-On with SAML 2.0 for Projects | Supabase Docs

<https://supabase.com/docs/guides/auth/enterprise-sso/auth-sso-saml>

6 7 Cron | Supabase Docs

<https://supabase.com/docs/guides/cron>

8 PGMQ Extension | Supabase Docs

<https://supabase.com/docs/guides/queues/pgmq>

9 10 37 Storage | Supabase Docs

<https://supabase.com/docs/guides/storage>

11 Connecting to Metabase | Supabase Docs

<https://supabase.com/docs/guides/database/metabase>

12 13 14 15 GitHub - crbnos/carbon: Carbon is an open source ERP, MES and QMS for manufacturing. Perfect for complex assembly, contract manufacturing, and configure to order manufacturing.

<https://github.com/crbnos/carbon>

16 17 GitHub - phamvuhoang/nextjs-supabase-boilerplate

<https://github.com/phamvuhoang/nextjs-supabase-boilerplate>

18 GitHub - akashmahlaz/School-Management-System: School ERP System | Next.js 15 + Supabase | Student Admissions | Class Management | Data Visualization (Recharts) | React Hook Form + Zod | Admin Dashboard | shadcn/ui | TypeScript

<https://github.com/akashmahlaz/School-Management-System>

19 20 GitHub - Saas-Starter-Kit/Saas-Kit-supabase: A template for building Software-as-a-Service (SAAS) apps with Reactjs, Nextjs and Supabase

<https://github.com/SaaS-Starter-Kit/SaaS-Kit-supabase>

21 GitHub - dzlau/stripe-supabase-saas-template

<https://github.com/dzlau/stripe-supabase-saas-template>

22 GitHub - scosman/CMSaaSStarter: A modern SaaS template/boilerplate built with SvelteKit, Tailwind, and Supabase. Includes marketing page, blog, subscriptions, auth, user dashboard, user settings, pricing page, and more.

<https://github.com/scosman/CMSaaSStarter>

23 GitHub - adrianhajdin/saas-template: SaaS Starter Template built with Next.js, Supabase & Clerk. Includes seamless auth, manage subscriptions and payments out of the box, and scale faster with a clean, reusable codebase — everything you need to kickstart your SaaS.

<https://github.com/adrianhajdin/saas-template>

24 GitHub - KolbySisk/next-supabase-stripe-starter: The highest quality SaaS starter with Next.js, Supabase, Stripe, and shadcn/ui

<https://github.com/KolbySisk/next-supabase-stripe-starter>

25 26 30 31 Database Architecture for Multi-Tenant Apps : r/Supabase

https://www.reddit.com/r/Supabase/comments/1ace4ag/database_architecture_for_multitenant_apps/

28 29 32 33 34 35 36 Best Practices for Supabase | Security, Scaling & Maintainability

<https://www.leanware.co/insights/supabase-best-practices>